# Question (1) // Theory part

#这里只是一部分而已，如果这个part想拿满分需要读多一点其他的

1. How object-oriented design works (3 step)
Step 1 - Identify the objects for the problem
Step 2 - Define data and operations for each object
Step 3 - Establish interactions between objects

2. Three Basic principles of OO design
Encapsulation
- Binding variables and methods into a single unit
- Data Hiding
Exp : private…. , protected…..

Inheritance
- The ability to create new classes from existing classes
- Promotes code reusability and establishes a “is-a” relationship.

Polymorphism
- The ability to use the same expression to denote different operations

3. Class VS Object
Class - A template or blueprint that defines the data fields and method an object will have.
Object - An object is an instance of a class.

4. Three roles of java identifiers
- Can only use letters, digits, underscores, and dollar signs
- Must start with a letter, an underscore, a dollar sign
- Can be of any length
- Cannot be a reserved word

Exp of valid identifiers
- Myvariable

Exp of invalid identifiers
- 123variable

5. Implicit casting VS Explicit casting
- Implicit casting is when you cast the data type to its higher range (upcasting)
Exp : int num = 10;  / double result = num;  //  int => double 向上兼容

- Explicit casting is when you cast the data type to its lower range (downcasting)
Exp : double value = 9.7; / int newValue = (int)value;  // double => int 向下兼容

6. Method signature
- Method signature is the combination of the method name and the parameter list.
Exp : max(int num1, int num2)

7. Formal parameters VS Actual parameters
Formal parameters - The variables defined in the method header are known as formal parameters.
Exp : max(int num1, int num2)

Actual parameters - When a method is invoked, pass a value to the parameter. This value call as an actual parameter.
Exp : int = max(x , y)

8. Method Overloading VS Method Overriding
Method Overloading - Two or more methods with the same name but different parameter list.

Method overriding occurs when a subclass defines a method with the same signature (same name, parameters, and return type) as a method in its superclass.

9. Public access VS Private access
Public access - Public access makes classes, methods, and data fields accessible from any class

Private access - Private access makes method and data fields accessible only from within its own class.

10. Procedural Paradigm (PP)  VS Object-Oriented paradigm (OOP)
PP
- Data and methods are loosely coupled.
- Software design focuses on designing methods

OOP
- Couples data and methods together into objects
- Software design focuses on objects and operations on objects

11. Accessor methods  VS Mutator methods
Accessor methods - Methods used to read private properties (AG)
Exp : getter or get method

Mutator methods -  Method used to modify private properties (MS)
Exp : setter or set method

12. Visibility modifiers
Private - The data or methods can be accessed only within the declaring class

Default - By default, the class, variable or method can be accessed by any class in the same package (folder)

Public - The class data or method is visible to any class in any package

13. Encapsulation (Principles 以外的题目问encapsulation就用这个)
- Class should use the private modifier to hide its data from direct access by clients.

14. This keyword (this.)
- The this keyword is the name of a reference that refers to a calling object itself.

# Question (2) // 给一个UML图，写一个完整的类 + driver测试程序

1. UML diagram basic

| **Employee (Class name)** |
| --- |
| + example1 : String (public)<br>- example2 : String (private)<br># example3 : String (protected)<br>- example4 : int (private static) |
| +Employee()  (no-parameterised constructor)<br>+Employee(example2 : String, ….)  (parameterised constructor )<br>+setter & getter  (看下面的format)<br>+toString() : String （看下面的format） |

(Exp : private String employeeID)
// getter format
public data type getvariable name () {
    return variable name;
}
Ans:
public String getEmployeeID(){  // 大字母开头
    return employeeID; }

// setter format
public void setvariable name (data type variable name) {
    this.variable name = variable name;
}
Ans:
public void setEmployeeID(String employeeID){   // 大字母开头
    this.employeeID = employeeID; }

// toString format (%s = string , %d = int, %.?f = float)
public String toString{
    return String.format("Employee ID: %s", employeeID); }

Practice 1

**Question 2**

Consider the following UML class diagram in *Figure 1*, which represents a class Employee and its details.

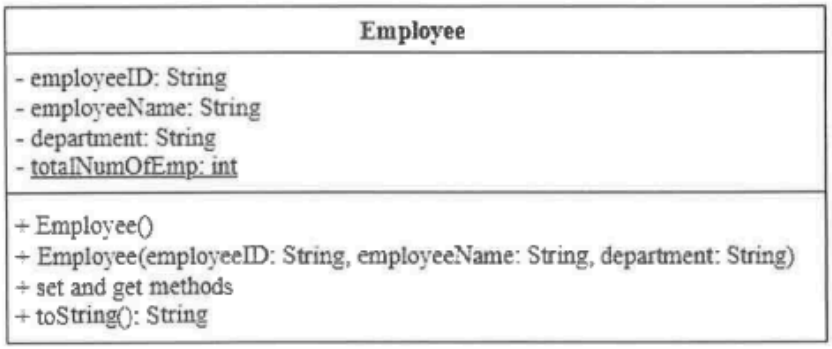



Figure 1: Class diagram

a) Based on *Figure 1*, construct a class named Employee with the following requirements:

- The constructors should increment the static variable totalNumOfEmp by 1 each time a new Employee object is created.
- Include set and get methods for all data members.
- Implement a toString() method that returns all information about an employee.

(16 marks)

(a) Ans

```
public class Employee {
    // data fields
    private String employeeID;
    private String employeeName;
    private String department;
    private static int totalNumOfEmp = 0;

    // no-parameterised constructor
    public Employee(){
    totalNumOfEmp++;
    };
    //parameterised constructor
    public Employee(String employeeID, String employeeName, String
    department){
    this.employeeID = employeeID;
    this.employeeName = employeeName;
```

```
        this.department = department;
        totalNumOfEmp++;
        }

        // getter
        public String getEmployeeID(){
        return employeeID;
        }
        public String getEmployeeName(){
        return employeeName;
        }
        public String getDepartment(){
        return department;
        }
        public static int getTotalNumOfEmp(){
        return totalNumOfEmp;
        }

        //setter
        public void setEmployeeID(String employeeID){
        this.employeeID = employeeID;
        }
        public void setEmployeeName(String employeeName){
        this.employeeName = employeeName;
        }
        public void setDepartment(String department){
        this.department = department;
        }

        // toString method
        public String toString(){
        return String.format("Employee ID: %s\nEmployeeName: %s\nDepartment:
        %s\nTotal Number Of Employee: %d",
        employeeID, employeeName, department, totalNumOfEmp);
        }
}
```

## 2. Driver program

这个step是我个人建议，不一定要跟
Step 1 - 创个Test? 的class
Step 2 - 里面在创个public static void main(String[] agrs)
Step 3 - main里面可以开始写program

#no-argument constructor/no-parameterised constructor
- 需要先create object / Exp: Employee employee1 = new Employee()
- 再用回去setter method 加东西下去 / Exp : employee1.setEmployeeID(??)

#parameterised constructor
- 直接用会之前create了的object / Exp : Employee employee2 = new Employee(???); // 里面放对应的parameter

Practice 1

b) Write a driver program to test the class that you have created in Question 2 a). Your program should perform the following tasks:

- Use the **no-argument constructor** to create an Employee object and set the values using set methods. Assign the following values:

| Employee ID | Employee Name | Department |
|---|---|---|
| E1001 | John Smith | Human Resources |

- Use the **parameterised constructor** to create another Employee object with the following values:

| Employee ID | Employee Name | Department |
|---|---|---|
| E1002 | Alice Brown | IT |

- Display the information of both Employee Objects, including the total number of employees. The sample output is shown in *Figure 2*.

```
General Output
--------------------Configuration: <Default>--------------------
Employee ID: E1001
Employee Name: John Smith
Department: Human Resources

Employee ID: E1002
Employee Name: Alice Brown
Department: IT

Total Number of Employees: 2

Process completed.
```

Figure 2: Sample Output

(9 marks)

(b) Ans

```
public class TestEmployee{
        public static void main(String[] agrs){

                // 用 no-arg constructor 创建对象，再用 setter 设值
                Employee employee1 = new Employee();
                employee1.setEmployeeID("E1001");
                employee1.setEmployeeName("John Smith");
                employee1.setDepartment("Human Resources");

                // 用 parameterised constructor 创建对象
                Employee employee2 = new Employee("E1002", "Alice Brown",
                "IT");

                // 可以有自己print的方式，不一定要跟着我
                // 用回直接的toString method (看题目要print什么样就跟着)
                System.out.println(employee1.toString());
                System.out.println();
                System.out.println(employee2.toString());
                System.out.println("Total Number Of Employee: " +
                Employee.getTotalNumOfEmp());
                System.out.println("Process completed.");
        }
}
```

# Question (3)
# //abstraction/inheritance/polymorphism + 子类实现 + 多态数组遍历+instanceof判断

# 先看题目跟UML，清楚是什么类型的class跟情况(e.g abstract , inheritance)
# 斜体字的 (italics word) 是abstract来的
#看下面的practice去理解
#最重要的是自己要多做就能理解了的

# Practice 1

WoodArt Furniture store builds and sells wooden tables. The store needs a system to record the details of each table made and to calculate its cost. The cost of a table depends on the material used and the size of the table. There are two types of tables made in the store: RectangleTable and RoundTable.

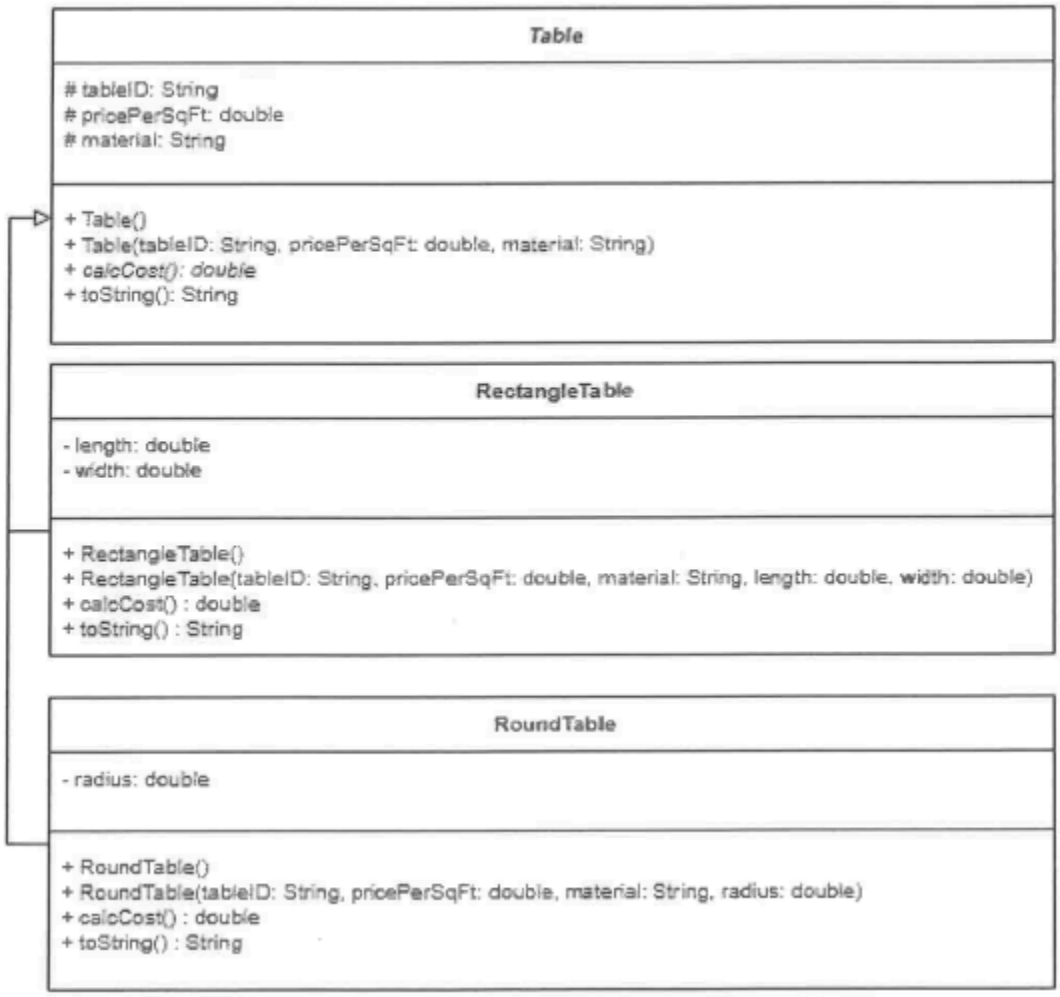



Figure 1: UML Class Diagram

通过UML，可以知道RoundTable & RectangleTable inheritance Table
然后Table 是 abstract class（Exp : *Table*）
还有abstract calcCost() method (Exp : *calcCost() : double*)

通过下面的题目，可以知道需要parameterised constructor with the maximum number of parameters.
然后各自的calcCost() method
还有toString() method 要returns all data member values of the Table.
还有Exclude all accessor and mutator methods.

**Question 3 (Continued)**

Each class must include the following:

- A parameterised constructor with the maximum number of parameters.
- A calcCost() method. The formulas to calculate the cost for both RectangleTable class and RoundTable class are shown below:
  - For RectangleTable class:
    cost = length × width × pricePerSqFt
  - For RoundTable class:
    cost = π × radius² × pricePerSqFt
- A toString() method that returns all data member values of the Table.

a) Construct an **abstract superclass** named Table that stores the common data members and methods.
Note: Exclude all accessor and mutator methods. (8 marks)

b) Construct a **subclass** named RectangleTable.
Note: Exclude all accessor and mutator methods. (9 marks)

(a) Ans

```
public abstract class Table{
        // data fields
        protected String tableID;
        protected double pricePerSqFt;
        protected String material;

        //parameterised constructor (maximum number of parameters)
        public Table(String tableID,double pricePerSqFt, String material){
        this.tableID = tableID;
        this.pricePerSqFt = pricePerSqFt;
        this.material = material; }

        // abstract method (一般情况下里面是没有东西的)
        public abstract double calcCost();

        // 父类的toString()
        public String toString() {
            return String.format(
                "Table ID: %s\nMaterial: %s\nPrice Per Sq Ft: RM%.2f",
                tableID, material, pricePerSqFt); }
}
```

(b) Ans

```
public class RectangleTable extends Table{
        private double length;
        private double width;

        public RectangleTable(){
        super(“ ”, 0.0, “ ”);
        }

        public RectangleTable(String tableID, double pricePerSqFt, String material,
        double length, double width){
        super(tableID, material, pricePerSqFt);
        this.length = length;
        this.width = width;
        }

        public double calcCost(){
        return length * width * pricePerSqFt;
        }

        // 先调用父类的toString(), 再追加自己的特有属性
        public String toString() {
        return super.toString() + String.format("\nLength: %.1f ft\nWidth: %.1f
        ft\nCost: RM%.2f", length, width, calcCost()); }
}
```

c) Assume that a **subclass** named RoundTable has been implemented based on the UML class diagram in *Figure 1*. Construct a driver program to test both the RectangleTable and RoundTable classes. In the program, declare a polymorphic array and instantiate a RectangleTable object and a RoundTable object using parameterised constructors. Using a for loop, determine the type of each object and print an appropriate label, such as "Rectangle Table" or "Round Table" followed by the table details, including the cost, as shown in *Figure 2*.

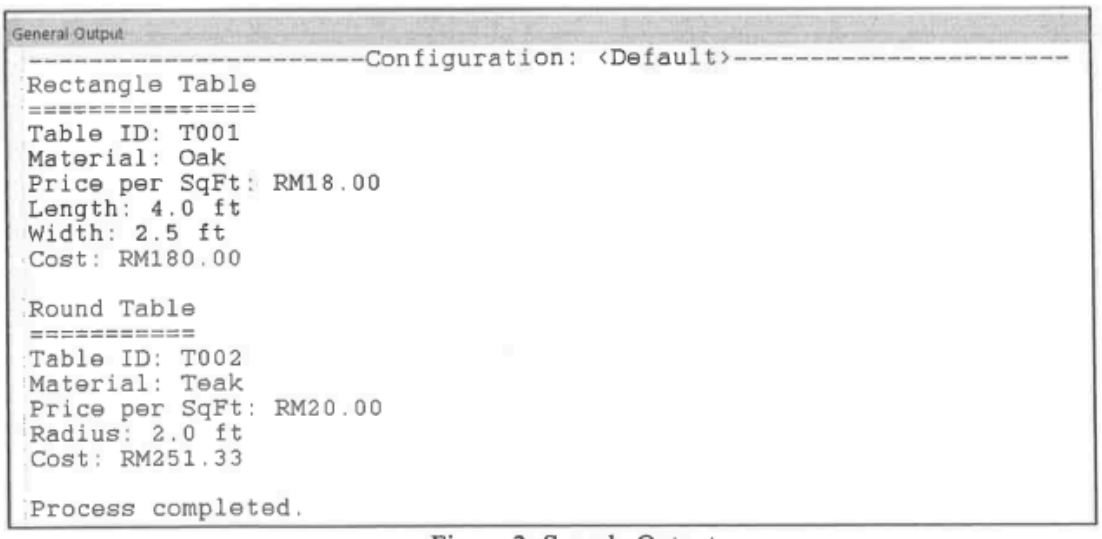



Figure 2: Sample Output

(8 marks)

通过题目可以知道

- Assume that a subclass named RoundTable has been implemented
- Construct a driver program to test both the RectangleTable and RoundTable classes. // create a Test? class
- Declare a polymorphic array and instantiate a RectangleTable object and a RoundTable object using parameterised constructors.

// polymorphic array(多态数组)

// 多态数组 = 用父类类型声明的数组，里面存放不同类型的子类对象。

- Using a for loop, determine the type of each object and print an appropriate label, such as "Rectangle Table" or "Round Table", followed by the table details including the cost, as shown in Figure 2.

// use for loop & instanceof去判断哪个是属于class

c) Ans

```
public class TestTable{
	public static void main(String[] args){

	// Step 1 & 2: 声明多态数组并实例化对象
	Table[ ] tables = new Table[2];
	tables[0] = new RectangleTable("T001", "Oak", 15.50, 6.0, 4.0);
	tables[1] = new RoundTable("T002", "Mahogany", 20.00, 3.5);

	// Step 3: using for loop / 我的习惯是用for loop，要用foreach也是可以的
	for(int i = 0; i < tables.length; i++){
		// Step 4: instanceof 判断类型
		if (tables[i] instanceof RectangleTable){
			System.out.println("Rectangle Table");
		}
		else if(tables[i] instanceof RoundTable){
			System.out.println("Round Table");
		}
		System.out.print("==============”);

		 // Step 5: 打印对象详情（含cost）
		System.out.println(tables[i].toString());
		System.out.println(); // next line
	}
	System.out.println("Process Completed");
	}

}
```

# Question (4) // interface题 + UML类关系题 + String输出题

#有很多种的题目，看中到哪个 // 我拿3个最常出的, *interface题必读*
#没有准确的ans，这些是我自己大概的ans ( Reference AI)

## 1. UML class diagram relationship

#实际上这些部分我也不是很懂，大致理解一点罢了

**Question 4**

a) In a Restaurant Food Ordering System, a Customer can place one or more Order. Each Order has at least one OrderDetail, which shows the quantity of each Menu ordered and any special remarks. The system includes two types of Menu, categorised as Food and Drink. A Menu can appear in many OrderDetail, but each OrderDetail belongs to only one Order and will be removed if the Order is cancelled.

Based on the scenario above, identify the classes involved in each of the following relationships and illustrate your answer using an UML class diagram for each relationship. In your diagram, clearly show the correct multiplicity (if applicable).

(i) Inheritance (2 marks)

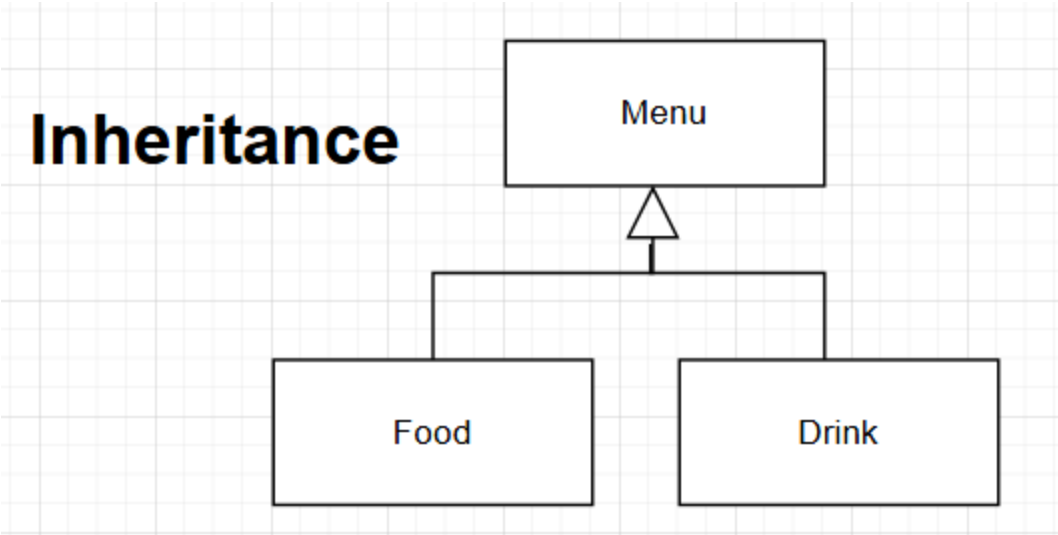
(ii) Association (2 marks)

(iii) Aggregation (2 marks)

(iv) Composition (2 marks)

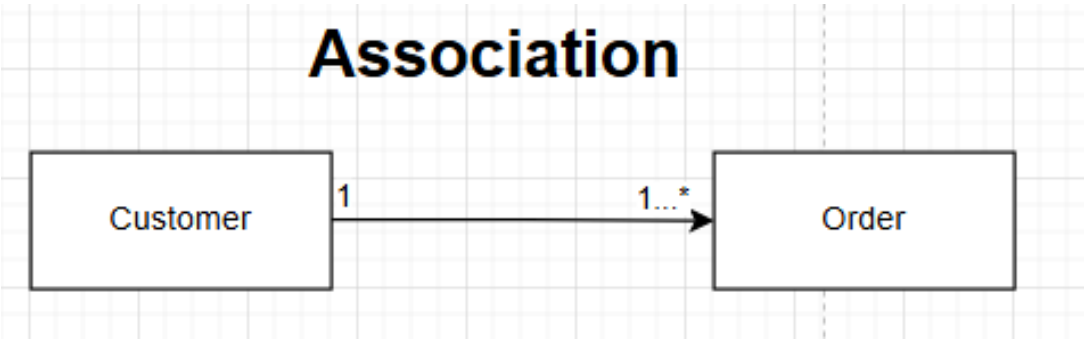
(i) Inheritance // keyword "categorised as"、"two types of"
题目有说
"Two types of Menu, categorised as Food and Drink"

Ans:
Inheritance means a child class is a type of the parent class. Here, Food and Drink are both types of Menu.



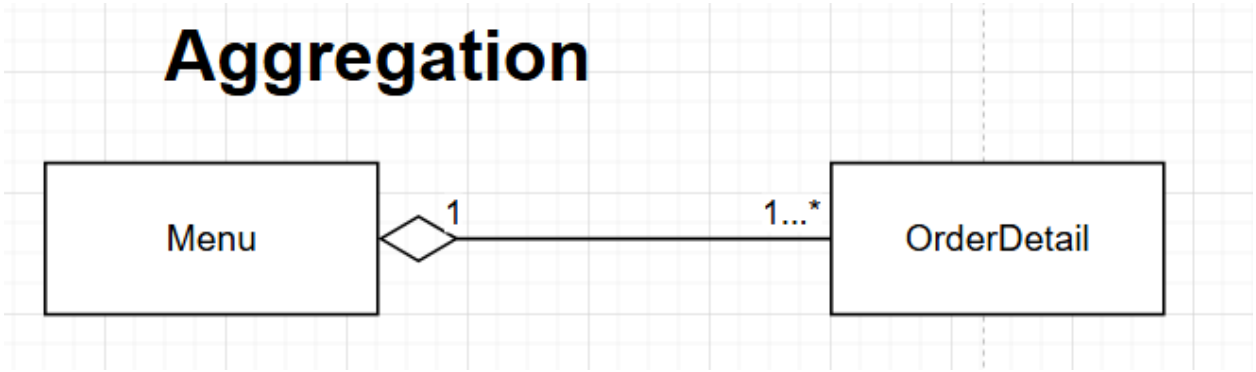

(ii) Association / keyword "uses"、"treats"、"可以独立存在"
题目有说
"Customer can place one or more Order"

Ans:
Association means two classes use or interact with each other. A Customer places one or more Orders. This is a basic relationship with multiplicity.



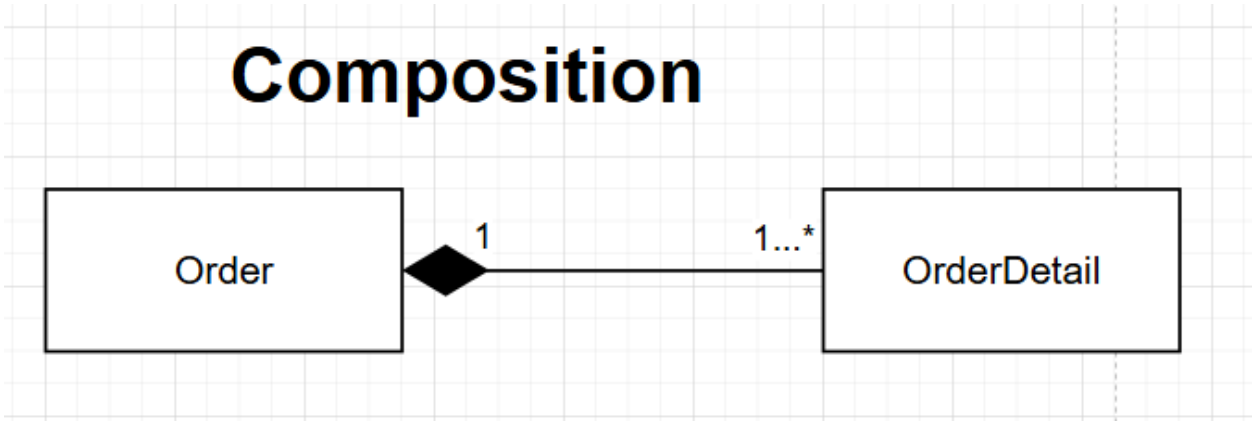
(iii) Aggregation / keyword "has"、"整体-部分但可独立存在"
题目有说
"A Menu can appear in many OrderDetail"

Ans:
Aggregation is a "has-a" relationship where the parts can exist independently. An Order has one or more OrderDetails, but the key here is that Menu can appear in many OrderDetails — Menu exists on its own.




(iv) Composition / keyword "由...组成"、"...被...拥有"、"被删除"
题目有说
"Each Order has at least one OrderDetail..."
"...each OrderDetail belongs to only one Order"
"...will be removed if the Order is cancelled"
Mean - OrderDetail 不能脱离 Order独立存在 → 强关系 → Composition

Ans:
Composition is a stronger "has-a" relationship where the parts cannot exist without the whole.



## 2. Interface

# Interface 就像一份"合约"，里面只写方法的名字，但不写方法的内容
# 任何 class 只要 `implements` 这个 interface，就必须把里面所有方法都写完整
# Interface 里的方法默认都是 `abstract`（抽象的），意思是没有方法体 `{}`
# 在 interface 里声明变量，它自动变成 `public static final` / 但是，还是写完整版的比较好
# `void`（没有返回值 ） // return什么就写什么 (Exp : if return double - double method();

Practice 1

b) (i) Construct an interface named PaymentStatus which contains **TWO (2)** abstract void methods named paymentSuccessful and paymentFailed. (3 marks)

(ii) Construct a class named Transaction that implements the PaymentStatus interface.

- The paymentSuccessful method should display the message:
  "Payment completed successfully. Thank you!"
- The paymentFailed method should display the message:
  "Payment failed. Please check your details and try again." (5 marks)

b) (i) Ans

```
public interface PaymentStatus {
   void paymentSuccessful();
   void paymentFailed();
}
```

b) (ii) Ans

```
public class Transaction implements PaymentStatus{
        public void paymentSuccessful(){
        System.out.println("Payment completed successfully. Thank you!");
        }

        public void paymentFailed(){
        System.out.println("Payment failed. Please check your details and try
        again."); }
}
```

Practice 2

**Question 4**

a) (i) Create an **interface** named Discountable, which declares a **constant variable** named DISCOUNT_RATE with the value 0.05 and a method named calcPriceAfterDiscount(), which returns a double value. (3 marks)

(ii) Construct a class named Item that implements the Discountable interface. The following should be included in the class:

- An instance variable named price.
- The calcPriceAfterDiscount() method should calculate and return the price after applying the discount using the formula:

price after discount = price × (1 - DISCOUNT_RATE)

(5 marks)

a) (i) Ans

```
public interface Discountable {
   // 完整版的比较好
   public static final double DISCOUNT_RATE = 0.05;
   double calcPriceAfterDiscount();    // abstract method, returns double
}
```

a) (ii) Ans

```
public class Item implements Discountable {
   double price;    // instance variable

   public double calcPriceAfterDiscount() {
      return price * (1 - DISCOUNT_RATE);
   }
}
```

# 3. Java String Methods

## Practice 1

c) Determine the output of the following statements (each statement is executed independently of the others):

```
String str = "Programming is fun!";
String s = new String("Programming is fun!");
```

(i) System.out.println(str.toUpperCase()); (1 mark)

(ii) System.out.println(str.substring(15, 18)); (1 mark)

(iii) System.out.println(str.indexOf("is")); (1 mark)

(iv) System.out.println(str.replace('i', '*')); (1 mark)

(v) System.out.println(str.charAt(3)); (1 mark)

(vi) System.out.println(str.length()); (1 mark)

(vii) System.out.println(str.compareTo(s)); (1 mark)

(viii) System.out.println(str == s); (1 mark)

(ix) System.out.println(str.equals(s)); (1 mark)

`str = "Programming is fun!"`，我们先数好每个字符的**index（位置）**，从 **0** 开始：

```
P  r  o  g  r  a  m  m  i  n  g     i  s     f  u  n  !
0  1  2  3  4  5  6  7  8  9  10 11 12 13 14 15 16 17 18
```

Ans:

(i) PROGRAMMING IS FUN! // 全部大写

(ii) fun // 从 index 15 取到 18（不包括18），即 index 15, 16, 17

(iii) 12 // 找 "is" 第一次出现的位置，index 12 是 i，index 13 是 s

(Extra) str.lastIndexOf('m'); // return最后一个 'm' 的位置

(iv) Programm*ng *s fun! // 把所有 i 换成 *

(v) g // index 3 的字符

(vi) 19 // 字符串总长度（包括空格和！）

(vii) 0 // 两个String内容一模一样，compareTo 返回差值

(viii) false // == 比较的是内存地址（reference），不是内容 str 是 String literal（存在 String Pool）s 是用 new 创建的（存在 Heap，不同地址）

(ix) true // .equals() 比较的是内容，两者内容相同

**重点总结**

| 方法 | 作用 |
| --- | --- |
| `toUpperCase()` | 全变大写 |
| `substring(a, b)` | 取 index a 到 b-1 |
| `indexOf("xx")` | 找第一次出现的位置 |
| `replace(a, b)` | 把所有 a 换成 b |
| `charAt(n)` | 取第 n 个字符 |
| `length()` | 字符串长度 |
| `compareTo(s)` | 内容一样→`0` |
| `==` | 比地址 ❌（内容相同也可能 false） |
| `.equals()` | 比内容 ✅ |